

DEEP LEARNING PROJECT

Road Damage Detection



Using YOLOv8 on the RDD2022 Dataset — trained end-to-end on Google Colab

RDD2022

Dataset

YOLOv8

Model

Object Detection

Task

Potholes & Cracks

Target

Spot, count, and localise road damage in photos using a single-stage CNN detector — reaching 0.378 mAP@0.5 across 4 damage classes

PRESENTED BY : TAMANNA SATSANGI
HUDA KHAN
VARALIKA SHARMA

The Problem



Manual Inspection

Cities inspect roads by hand — slow, expensive, and inconsistent. Hard to scale across thousands of km.



Camera Coverage

Road cameras & dashcams generate millions of images. No human team can review them in real-time.



ML Solution

Train a YOLOv8 neural network on thousands of labelled photos to detect potholes and cracks automatically — fast, cheap, and scalable to an entire city's road network.

Goal: Mark every pothole & crack with a bounding box, classify it (D00/D10/D20/D40), and count damage instances per image — turning raw road photos into an inspection report automatically

RDD2022 Dataset

Dataset Overview

Country Packs

India + Japan

India images

~4,000 images

Japan images

~7,000 images

Damage Classes

D00 · D10 · D20 · D40

Format

JPEG + XML annotation

Split

80 % train / 20 % val



India — Varied terrain



Japan — High-quality roads

Damage Classes

D00

Longitudinal Crack

D10

Transverse Crack

D20

Alligator Crack

D40

Pothole

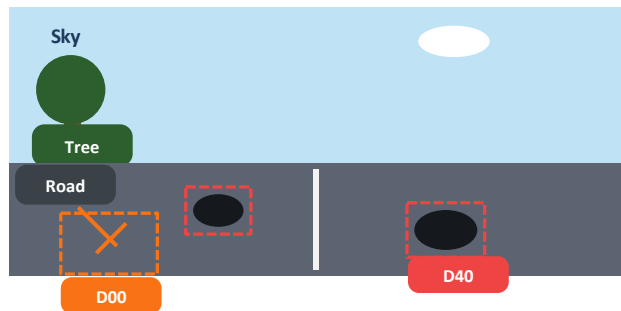


oad_crack_simulator (1).html

What Happens Inside One Image?

Suppose your first training image contains:

Image Contents



annotation.xml (Ground Truth)

```
<object>
  <name>D40</name>
  <bndbox>
    <xmin>120</xmin> <ymin>300</ymin>
    <xmax>210</xmax> <ymax>380</ymax>
  </bndbox>
</object>
```

Key Insight

The XML annotation tells the model EXACTLY where each pothole is located (x,y,w,h) and what class it belongs to (D40). During training the model compares its own prediction against these boxes and adjusts its weights to get closer.

Training Process

1

Image fed in

CNN knows nothing yet

2

Random Weights

Random prediction

3

Ground Truth check

Label says "D40" at x,y,w,h

4

Compute Loss

Objectness + Box + Class loss

5

Backpropagation

Gradients flow backwards

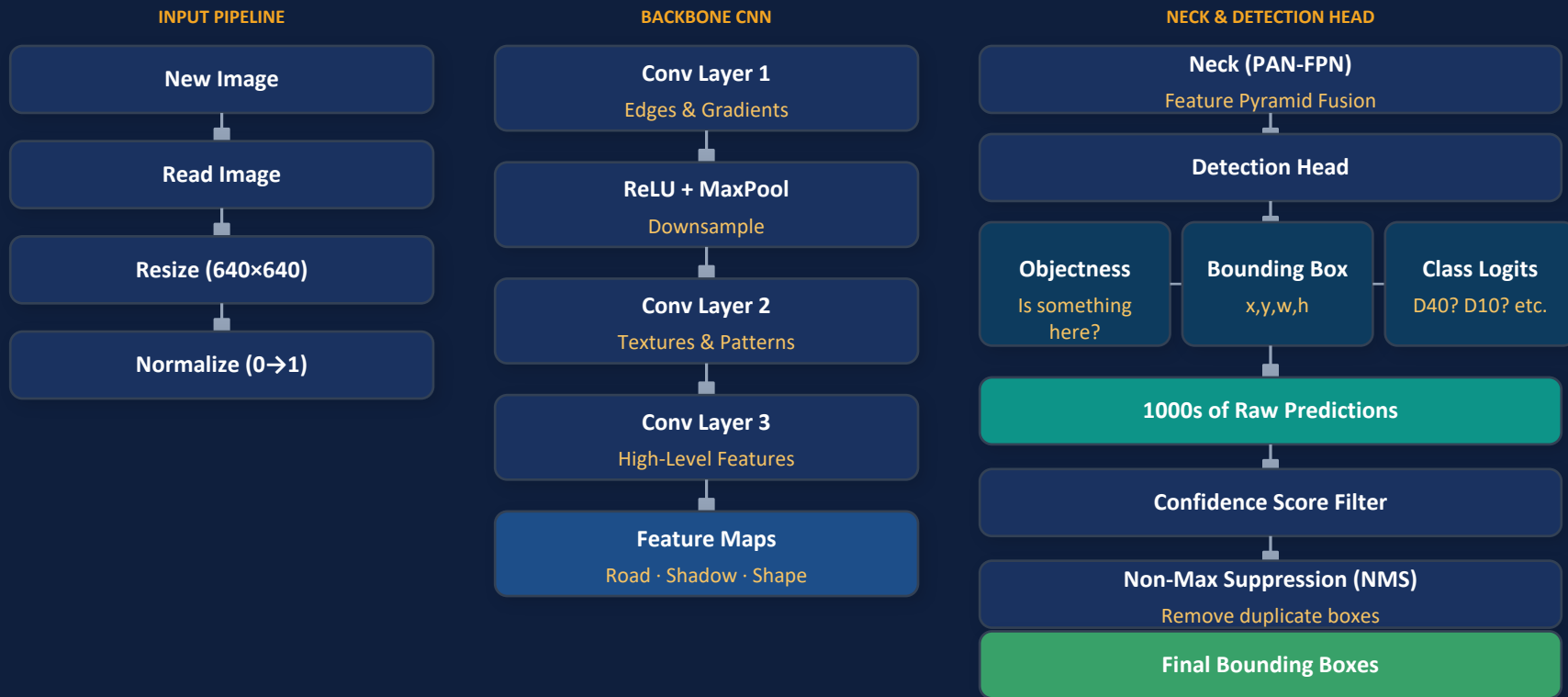
6

Weights Updated

$w \leftarrow w - \eta \times \partial L / \partial w$

↺ Repeat for thousands of images × 20 epochs → CNN learns pothole features → Save best.pt

YOLOv8 Complete Architecture



Training vs Inference



TRAINING

1. Image + Annotation (XML)
2. Resize → Normalize → Augment
3. Forward Pass through YOLOv8
4. Compare prediction with GT boxes
5. Compute YOLO Loss (3 components)
6. Backpropagation (chain rule)
7. Update all weights
8. Repeat × 20 epochs → best.pt

✓ Has loss ✓ Has backprop ✓ Needs labels



INFERENCE

1. New Image (no annotation needed)
2. Resize → Normalize
3. Forward Pass ONLY (no backprop)
4. Backbone → Neck → Detection Head
5. Thousands of raw box predictions
6. Filter by confidence score (≥ 0.25)
7. Non-Maximum Suppression (NMS)
8. Display final bounding boxes

X No loss X No backprop X No labels needed

YOLO Loss Function



Objectness Loss

$$\text{BCE}(\sigma(\text{obj_pred}), \text{has_object})$$

Is there an object in this grid cell at all? Binary yes/no. Penalizes cells that miss an existing object or claim objects where none exist.



Bounding Box Loss

$$\text{CIoU}(\text{pred_box}, \text{gt_box})$$

How well does the predicted box overlap with the ground-truth box? Uses Complete-IoU to penalize position, size, and aspect-ratio errors together.



Classification Loss

$$\text{BCE}(\sigma(\text{class_pred}), \text{one_hot_label})$$

What type of damage is it — D40 pothole, D10 crack, etc.? Binary cross-entropy across all classes with a soft one-hot label vector.

$$\text{Total Loss} = \lambda_1 \times \text{Objectness Loss} + \lambda_2 \times \text{Box Loss} + \lambda_3 \times \text{Class Loss}$$

Loss Function — Worked Calculations

L_obj (Objectness)

Binary Cross-Entropy

$$-[t \cdot \log(p) + (1-t) \cdot \log(1-p)]$$

Positive cell (t=1, p=0.72)

$$-\log(0.72) = 0.3285$$

Negative cell (t=0, p=0.15)

$$-\log(0.85) = 0.1625$$

$$L_{obj} = 0.2455$$

L_cls (Classification)

BCE per class (D00–D40)

$$L = (1/4) \cdot \sum_c BCE_c$$

GT: D40 pothole

$$D00: p=0.10 \rightarrow 0.1054$$

$$D10: p=0.05 \rightarrow 0.0513$$

$$D20: p=0.20 \rightarrow 0.2231$$

$$D40: p=0.65 \rightarrow 0.4308$$

$$L_{cls} = 0.2026$$

L_box (IoU Regression)

L_box = 1 - IoU

$$IoU = \text{Intersection} / \text{Union}$$

$$\text{Pred [100,200,180,270]}$$

$$\text{GT [110,195,190,275]}$$

$$\text{Inter} = 4900 \text{ px}^2$$

$$\text{Union} = 7100 \text{ px}^2$$

$$IoU = 4900 / 7100 = 0.690$$

$$L_{box} = 1 - 0.690 = 0.310$$

$$L_{total} = 7.5 \times L_{box} + 0.5 \times L_{obj} + 0.5 \times L_{cls} = 7.5 \times 0.310 + 0.5 \times 0.245 + 0.5 \times 0.203 = 2.548$$

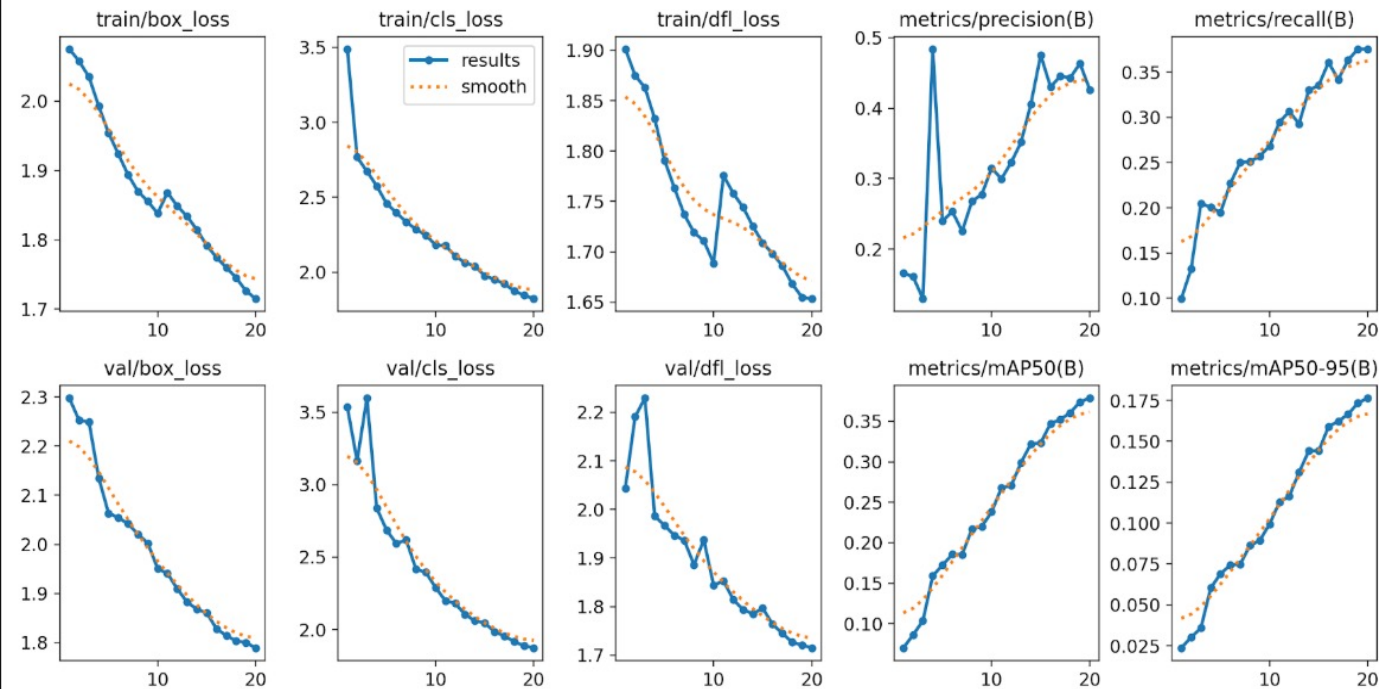
End-to-End Workflow



Output: Bounding-boxed image with damage count | Metric: mAP (higher = better)

Training Curves — All 20 Epochs (Real Data)

Box loss, classification loss, DFL loss all decreasing; mAP rising — model was still learning at epoch 20



What Each Panel Means

box_loss ↓ Bounding box coord error shrinking — model learns where damage is

cls_loss ↓ Classification error dropping — model learns D00/D10/D20/D40 apart

dfl_loss ↓ Distribution Focal Loss — fine-grained box edge prediction improving

precision ↑ Noisy but trending up; ~0.45 by ep20 — boxes drawn are mostly correct

recall ↑ Rising cleanly to ~0.38 — more real damage being found each epoch

mAP50 ↑ Validation mAP reaching ~0.37 — model had NOT plateaued yet!

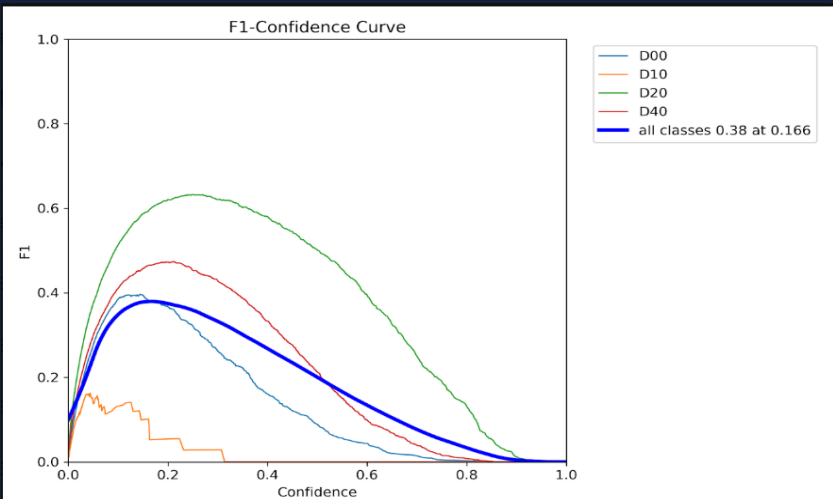
mAP50-95 ↑ Strict metric (0.025→0.175) — consistent improvement throughout

KEY FINDING: val/box_loss and mAP50 curves show the model had NOT converged at epoch 20. Running 50–100 epochs on Colab would likely push mAP@50 past 0.50 for D20 and 0.45+ overall.

F1 & Precision vs Confidence Threshold

Choosing the right confidence cut-off balances how many boxes are drawn against how accurate they are

F1-Confidence Curve



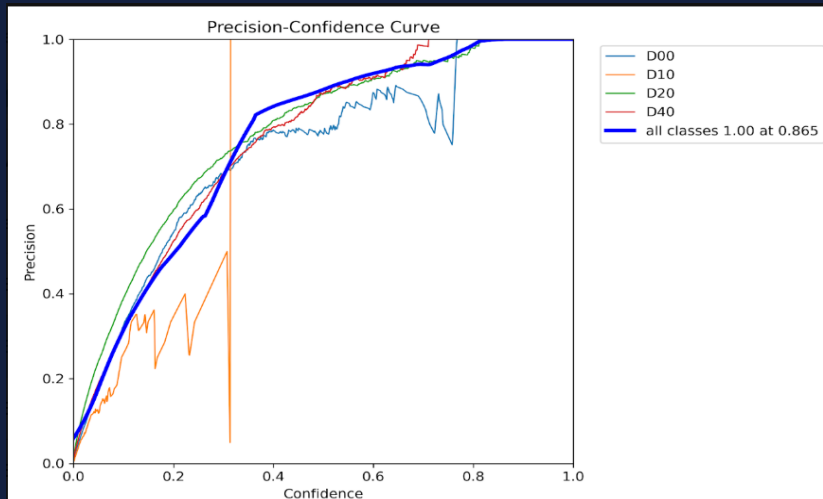
F1 peaks at 0.166 conf

Best overall balance at very low confidence (0.166). Beyond ~0.4, F1 drops sharply as recall collapses.

D20 highest F1 (~0.63)

Alligator cracks are easiest — wide area, distinctive texture. Model is confident and correct.

Precision-Confidence Curve



D10 lowest F1 (~0.11)

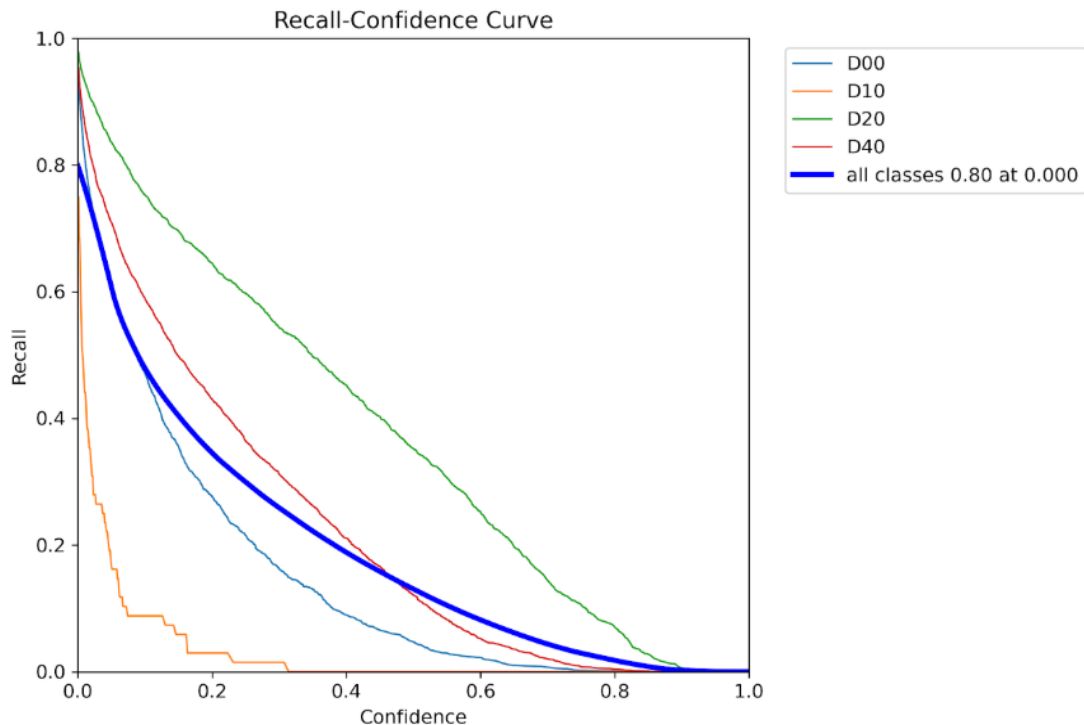
Transverse cracks are very narrow, few pixels. Model is unsure and often wrong → poor F1.

Precision = 1.0 @ 0.865

At conf ≥ 0.87, every box drawn is correct (precision=1.0) — but barely anything is detected.

Recall vs Confidence — How Much Damage Is Found?

Recall drops as confidence threshold rises — stricter cut-off means fewer detections, more misses



Reading This Curve

All classes @ conf=0

Recall = 0.80 — model FINDS 80% of all damage if we keep every box

D20 best (green)

Still ~0.60 recall at conf=0.4 — D20 is consistently detected with high confidence

D10 worst (orange)

Recall collapses to 0 by conf=0.15 — model has very low confidence for D10

D00 (blue) below avg

Longitudinal cracks drop fast — narrow lines detected inconsistently

D40 (red) middle

Potholes hold recall well until ~0.6 confidence threshold

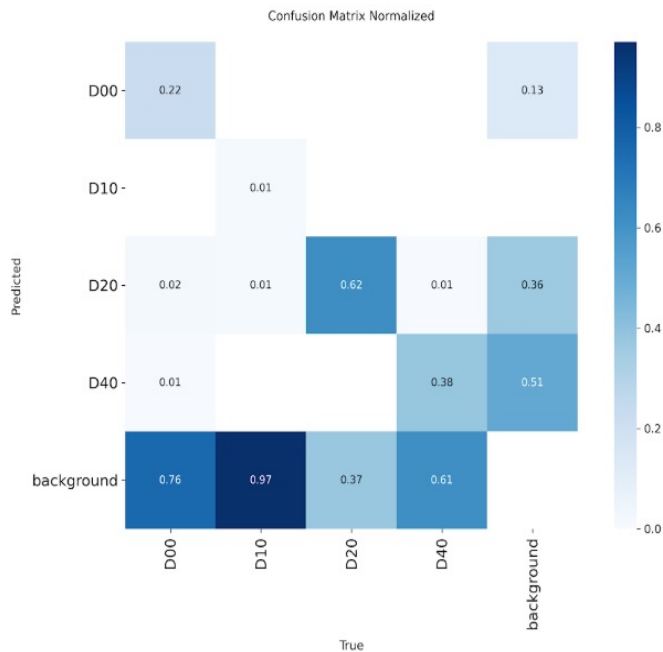
Trade-off

Lower conf → higher recall but more false positives
Higher conf → fewer boxes but more precise

PRACTICAL USE: For a road inspection system, use conf=0.15–0.20 to catch as many real cracks as possible, then have a human verify.

Confusion Matrix — What Gets Confused With What?

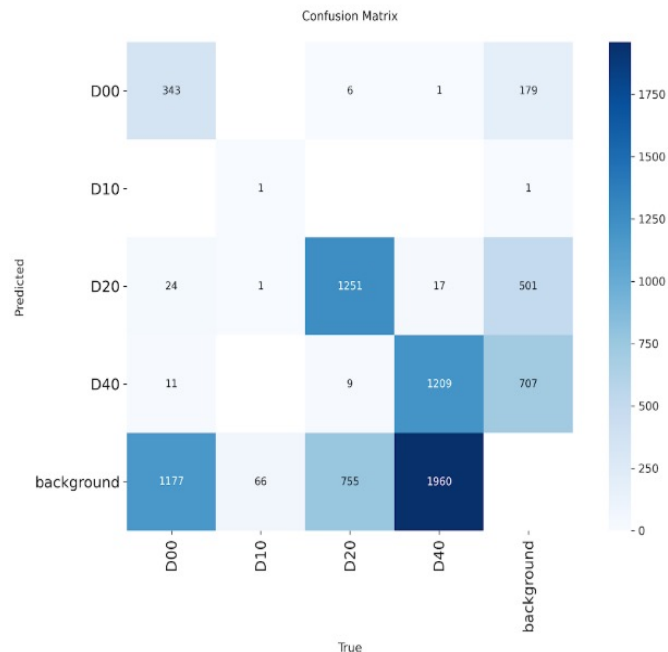
Normalised (%) left | Raw counts right — both from your actual validation run



Normalised

D00: 22% correct | 76% miss as BG

D10: 1% correct | 97% miss as BG



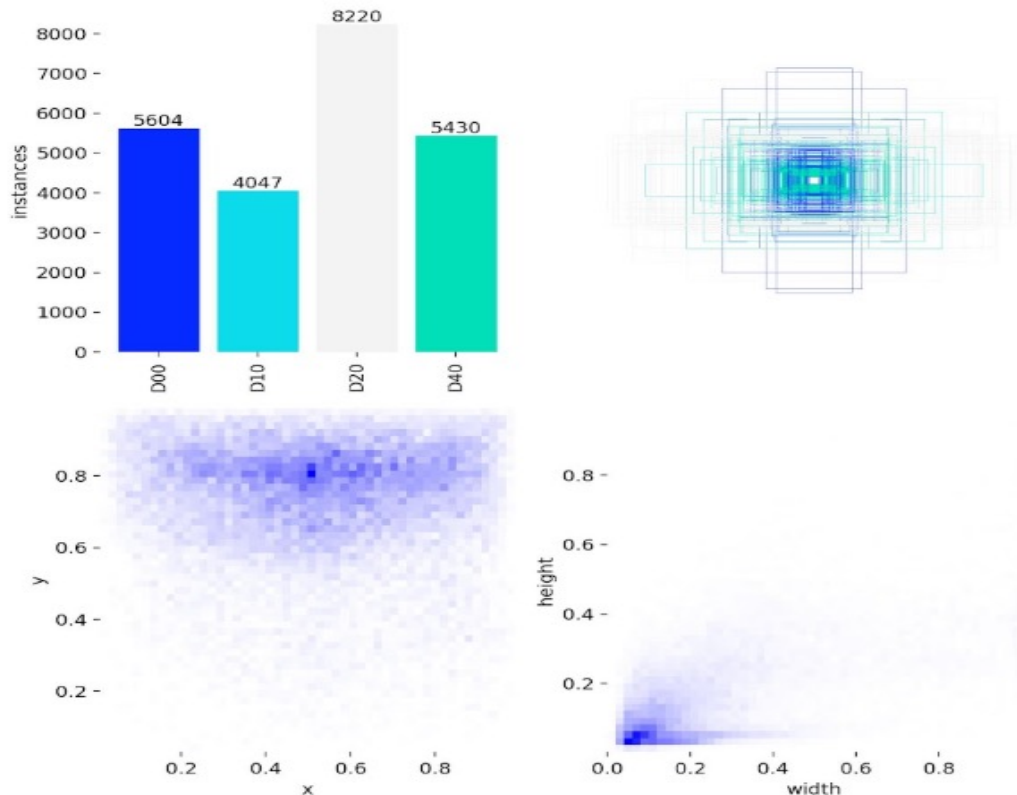
Raw Counts

D20: 62% correct | 37% miss as BG

D40: 38% correct | 61% miss as BG

Dataset Distribution & Label Analysis

Understanding the data imbalance that explains many model weaknesses



What The Charts Tell Us

Class Counts (bar chart)

D20 has 8,220 instances — most common.

D00: 5,604 D40: 5,430 D10: 4,047

→ D10 fewest = model sees less = worst AP (0.076)

Box position (heatmap)

Most damage concentrated in center/lower frame

— road center where camera points.

Good: model learns to focus there.

Box size (scatter)

Almost all boxes are small ($\text{width} + \text{height} < 0.2$)

— confirms why small object detection is hard.

YOLO P3 head (80×80) handles these.

Box shape (grid)

Boxes cluster around center, various aspect ratios

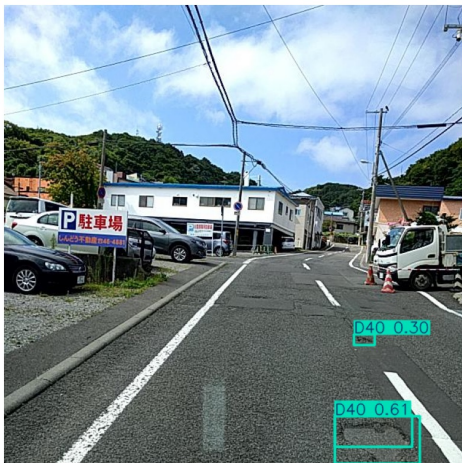
— D20 alligator patches are nearly square;

D00 cracks are wide and short.

Japan_007237.jpg



Detected Image



Input Image
(600×600×3)



Grayscale
(600×600)



Convolution
(598×598)



ReLU
(598×598)



Max Pooling
(299×299)



Flatten
(89401,)



Fully Connected Layer



Logits
(4,)



Sigmoid



Confidence Scores

Flatten Layer Representation

After pooling you have

299 × 299

Imagine only a small example:

Before Flatten

12	18	20
25	35	40
10	15	28

Shape

3 × 3

↓

Flatten

[12, 18, 20, 25, 35, 40, 10, 15, 28]

Shape

9

Your actual case:

299×299

↓

89401

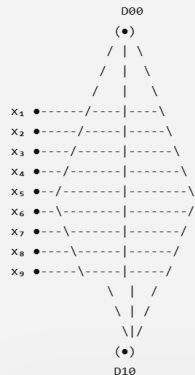
Fully Connected Layer Representation

Suppose there are only 9 inputs (instead of 89,401).

Flatten Vector

x_1
 x_2
 x_3
 x_4
 x_5
 x_6
 x_7
 x_8
 x_9

Now connect them to 4 output neurons.



Similarly for D20 and D40

In reality,

89401 inputs

connect to

4 output neurons

This is why it is called **Fully Connected**.

Matrix Representation

Your flatten vector

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{89401} \end{bmatrix}$$

Shape

89401 × 1

Weights

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ \vdots & \vdots & \vdots & \vdots \\ w_{89401,1} & w_{89401,2} & w_{89401,3} & w_{89401,4} \end{bmatrix}$$

Shape

89401 × 4

Bias

$$b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

Shape

4 × 1

The FC layer performs

$$Z = X^T W + b$$

where

- X = Flatten vector
- W = Weight matrix
- b = Bias vector
- Z = Logits

Output

D00 = 2.3

D10 = -1.2

D20 = 3.7

D40 = 0.8

Sigmoid

D00 = 0.62

D10 = 0.23

D20 = 0.91

D40 = 0.69

These are called **logits**.

FORMULAE USED :-

1. RGB Image Representation

An RGB image is represented as a 3D matrix.

Image Shape = $H \times W \times 3$

Example:

$600 \times 600 \times 3$

where

- H = Height
- W = Width
- 3 = Red, Green, Blue

4. Convolution

The most important CNN operation.

For a kernel of size

$K \times K$

the convolution output is

$$Y(i,j) = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} X(i+m,j+n) \times W(m,n)$$

Example

Image

$$\begin{bmatrix} 224 & 223 & 220 \\ 220 & 219 & 180 \\ 215 & 180 & 120 \end{bmatrix}$$

Kernel

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Output

$$\begin{aligned} &224(1) + 223(0) + 220(-1) \\ &+ 220(1) + 219(0) + 180(-1) \\ &+ 215(1) + 180(0) + 120(-1) \\ &= 139 \end{aligned}$$

2. Grayscale Conversion

Converts three channels into one.

$Gray = 0.299R + 0.587G + 0.114B$

Example

$$\begin{aligned} R &= 217, \quad G = 227, \quad B = 228 \\ Gray &= 0.299(217) + 0.587(227) + 0.114(228) \\ &= 64.883 + 133.249 + 25.992 \\ &= 224.124 \approx 224 \end{aligned}$$

5. Feature Map Size

Without padding

$Output = N - K + 1$

where

- N = Input size
- K = Kernel size

Example

$600 - 3 + 1 = 598$

Output

598×598

General formula

$Output = \frac{N - K + 2P}{S} + 1$

where

- P = Padding
- S = Stride

6. ReLU Activation

Removes negative values.

$ReLU(x) = \max(0, x)$

Example

Input

$[-20, 35, -10, 50]$

Output

$[0, 35, 0, 50]$

3. Image Normalization (used in YOLO)

Converts pixel values from 0–255 to 0–1.

$I_{norm} = \frac{I}{255}$

Example

224

$\frac{224}{255} = 0.878$

7. Max Pooling

For every pooling window

$Output = \max(Window)$

Example

$$\begin{bmatrix} 20 & 80 \\ 35 & 40 \end{bmatrix}$$

↓

80

8. Pooling Output Size

General formula

$Output = \frac{N - F}{S} + 1$

Example

Input

598

Filter

2

Stride

2

Output

$\frac{598 - 2}{2} + 1 = 299$

9. Flatten

Converts

$H \times W$

into

$H \times W$

elements.

Example

$299 \times 299 = 89401$

10. Fully Connected Layer

The Dense layer performs

$Z = XW + b$

where

- X = Flatten vector
- W = Weight matrix
- b = Bias

For one neuron

$$z = \sum_{i=1}^n x_i w_i + b$$

Example

$z = x_1w_1 + x_2w_2 + \dots + x_nw_n + b$

11. Sigmoid Activation

Converts logits into probabilities.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Example

$\sigma(2) = 0.88$

13. Binary Cross Entropy Loss

Used with Sigmoid.

$$L = -y \log(p) - (1 - y) \log(1 - p)$$

where

- y = Ground Truth
- p = Prediction

15. Confidence Score (YOLO)

$$Confidence = Objectness \times Class\ Probability$$

Example

Objectness

0.90

Class Probability

0.80

Confidence

$$0.90 \times 0.80 = 0.72$$

16. Threshold

Decision Rule

$$Confidence \geq Threshold$$

Keep

Otherwise

Discard

17. Bounding Box Representation

YOLO predicts

$$(x, y, w, h)$$

where

- x = center x
- y = center y
- w = width
- h = height

18. Bounding Box Area

$$Area = w \times h$$

19. Intersection over Union (IoU)

One of the most important formulas.

$$IoU = \frac{Area\ of\ Intersection}{Area\ of\ Union}$$

where

$$Union = Prediction + GroundTruth - Intersection$$

20. Precision

$$Precision = \frac{TP}{TP + FP}$$

Measures how many predicted detections are correct.

21. Recall

$$Recall = \frac{TP}{TP + FN}$$

Measures how many actual objects were detected.

22. F1 Score

$$F1 = \frac{2PR}{P + R}$$

where

- P = Precision
- R = Recall

23. Average Precision (AP)

Area under the Precision–Recall Curve.

$$AP = \int_0^1 Precision(Recall) dRecall$$

24. Mean Average Precision (mAP)

Average AP over all classes.

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

25. mAP@50

Uses

$$IoU = 0.50$$

A prediction is considered correct if

$$IoU \geq 0.5$$

26. mAP@50–95

Average over multiple IoU thresholds:

$$0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95$$

Understanding mAP (Understanding-Check 1)

mAP = Mean Average Precision — the primary metric for object detection quality

Precision

Of all boxes the model drew, how many were actually over a real damage? $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Recall

Of all real damage spots in the image, how many did the model find? $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

IoU Threshold

A predicted box 'counts' as correct only if it overlaps the ground-truth box by $\geq 50\%$ ($\text{IoU} \geq 0.5$). Tiny cracks need very precise box placement.

Average Precision (AP)

Area under the Precision-Recall curve for one class. A perfect detector has $\text{AP} = 1.0$.

mAP

Mean of AP across all damage classes (D00, D10, D20, D40). A single number that summarises overall quality.

Why tiny cracks are hard

Small cracks occupy very few pixels. The box must be placed very accurately (high IoU) or the prediction is counted as a false positive — hurting mAP.

Where It Works & Where It Fails (Understanding-Check 2)

✓ Works Well

Photo 1: Clear daylight photo

Why: Pothole clearly lit; shadow creates depth cue model has learned.

Photo 2: Single large pothole

Why: High IoU easy to achieve on big object. High confidence score.

Photo 3: Dry, clean road surface

Why: Texture contrast between pothole and intact asphalt is high.

✗ Fails / Misses

Photo 1: Night-time image

Why: Low contrast and underexposure; features model learned (texture, shadow) are invisible.

Photo 2: Water-filled pothole

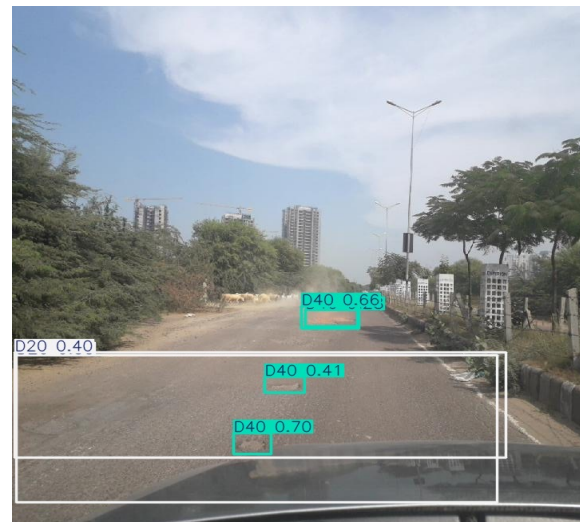
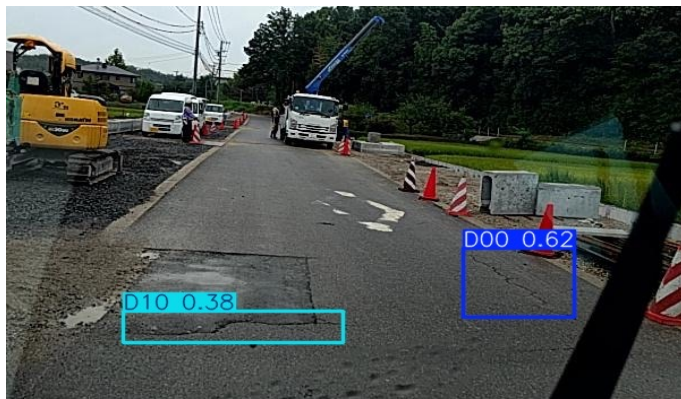
Why: Reflective water surface masks the depth cavity; looks like normal road at prediction time.

Photo 3: Hairline crack (D00)

Why: Occupies only a few pixels. Even a slightly off box fails $\text{IoU} \geq 0.5$ — counts as FP.

Detections in Action- Success

Trained model run on unseen dashcam frames — bounding boxes show predicted damage class + confidence score.



Detections in Action- Failure

Trained model run on unseen dashcam frames — bounding boxes show wrong predictions.





Q Commands + Code + Text ▶ Run all

Connect T4 ^



```
[ ] !pip install -q ultralytics
```

```
[ ] from ultralytics import YOLO
    from google.colab import files
    import matplotlib.pyplot as plt
    import cv2
```

```
[ ] from google.colab import drive
    drive.mount('/content/drive')
```

```
[ ] !find /content/drive -name "best.pt"
```

```
[ ] from ultralytics import YOLO

    model = YOLO("/content/drive/MyDrive/RoadDamage_Final_Results/detect/train-2/weights/best.pt")
```

```
[ ] from google.colab import files
```



Setup & Deliverables

Quick Start (5 steps)

- 1 Open Google Colab → New Notebook → GPU On
- 2 `pip install ultralytics`
- 3 Download RDD2022 India + Japan packs
- 4 Convert XML → YOLO txt format
- 5 `yolo detect train data=rdd.yaml model=yolov8n.pt epochs=20`

What to Hand In

-  Google Colab notebook (runs top-to-bottom)
-  Training loss graph (loss going down)
-  6–8 example photos with boxes drawn
-  mAP score printed by YOLO
-  Understanding-check 1: explain mAP
-  Understanding-check 2: 3 works + 3 fails
-  GitHub repo + README + 2-3 min demo video

Road Damage Detection

YOLOv8 × RDD2022

- ① Single-stage CNN detection: one forward pass finds all potholes
- ② Loss = Objectness + Box + Class → Backprop → Weights update
- ③ mAP measures quality: higher = better, tiny cracks are hardest
- ④ Free GPU on Google Colab — training takes ~2 hours

Start Building →